

Instituto Superior de Engenharia de Lisboa
Licenciatura em Engenharia Informática e de Computadores
Licenciatura em Engenharia Informática, Redes e Telecomunicações
Introdução à Programação na Web / Programação na Internet
Inverno de 2021/2022
Teste Global de Época Normal (8 de fevereiro de 2022) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 1 [7,5 valores]

Nas seguintes questões, selecione a **única** opção verdadeira. Uma resposta correta conta 1,25 valores; uma resposta incorreta desconta 0,25 valores ao total do grupo; uma questão não respondida conta 0 valores.

1. [1,25] Uma aplicação *web* recebe um pedido que não pode processar, uma vez que deteta a falta de um parâmetro obrigatório na *query string*. O *status code* adequado para responder a esta situação é:
A. 200
B. 300
C. 400
D. 500
2. [1,25] Num formulário HTML (criado com o elemento `form`) com submissão automática do pedido pelo botão de *submit* (definido como elemento integrante do formulário), este botão de submissão é definido com:
A. elemento `button` com atributo `type` de valor `post` (e `method` no `form` com valor `submit`)
B. elemento `submit` com atributo `type` de valor `button` (e `method` no `form` com valor `post`)
C. elemento `input` com atributo `type` de valor `submit` (e `method` no `form` com valor `post`)
D. elemento `request` com atributo `type` de valor `post` (e `method` no `form` com valor `submit`)
3. [1,25] Em código JavaScript executado pelo *browser* no âmbito de um documento HTML, a forma correcta de obter referências para os elementos cujo atributo `class` inclua *note* é:
A. `document.querySelector("#note")`
B. `document.querySelectorNote("class")`
C. `document.querySelectorAll(".note")`
D. `document.querySelector("class=note")`
4. [1,25] Considere o programa em JavaScript apresentado ao lado, que é executado via Node.js. Aproximadamente quantos segundos decorrem entre a afixação na consola de *START* e *DONE*?
A. 2 ou menos segundos
B. 3 segundos
C. 4 segundos
D. 5 ou mais segundos

```
let value = 0;
setTimeout(() => { value = 1 }, 1000);
console.log("START");
setTimeout(() => { value = 2 }, 3000);
while (value !== 2) /* do nothing */ ;
console.log("DONE!");
```
5. [1,25] Numa aplicação *web*, construída em Node.js utilizando o módulo *express*, existem recursos designados como *item*, identificados por URIs com path `/items/_id_`, em que `_id_` é o identificador de cada item. Pretende-se aceitar um pedido HTTP para criar um novo recurso *item*. Qual das seguintes configurações da aplicação *express* (app) é inaceitável de acordo com a especificação HTTP?
A. `app.get("/item/:id/create", createItemWithId)`
B. `app.put("/items/:id", createItemWithId)`
C. `app.post("/items", createItemWithId)`
D. `app.post("/items/:id", createItemWithId)`
6. [1,25] Uma função tem que obrigatoriamente ter na assinatura a palavra reservada *async*, se:
A. retornar uma *Promise*
B. no corpo da função for usada a palavra reservada *await*.
C. a função chamar outra função que também é *async*.
D. se não tiver como último argumento uma função de *callback*

Instituto Superior de Engenharia de Lisboa
Licenciatura em Engenharia Informática e de Computadores
Licenciatura em Engenharia Informática, Redes e Telecomunicações
Introdução à Programação na Web / Programação na Internet
Inverno de 2021/2022
Teste Global de Época Normal (8 de fevereiro de 2022) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 1 [7,5 valores]

Nas seguintes questões, selecione a **única** opção verdadeira. Uma resposta correta conta 1,25 valores; uma resposta incorreta desconta 0,25 valores ao total do grupo; uma questão não respondida conta 0 valores.

1. [1,25] Em código JavaScript executado pelo *browser* no âmbito de um documento HTML, a forma correcta de obter referências para os elementos cujo atributo *class* inclua *note* é:
 - A. `document.querySelectorNote("class")`
 - B. `document.querySelectorAll(".note")`
 - C. `document.querySelector("class=note")`
 - D. `document.querySelector("#note")`
2. [1,25] Considere o programa em JavaScript apresentado ao lado, que é executado via Node.js.
Aproximadamente quantos segundos decorrem entre a afixação na consola de *START* e *DONE*?

```
let value = 0;
setTimeout(() => { value = 1 }, 1000);
console.log("START");
setTimeout(() => { value = 2 }, 3000);
while (value != 2) /* do nothing */ ;
console.log("DONE!");
```

 - A. 2 ou menos segundos
 - B. 3 segundos
 - C. 4 segundos
 - D. 5 ou mais segundos
3. [1,25] Numa aplicação *web*, construída em Node.js utilizando o módulo *express*, existem recursos designados como *item*, identificados por URIs com path `/items/_id_`, em que `_id_` é o identificador de cada item. Pretende-se aceitar um pedido HTTP para criar um novo recurso *item*. Qual das seguintes configurações da aplicação *express* (app) é inaceitável de acordo com a especificação HTTP?
 - A. `app.put("/items/:id", createItemWithId)`
 - B. `app.post("/items", createItemWithId)`
 - C. `app.post("/items/:id", createItemWithId)`
 - D. `app.get("/item/:id/create", createItemWithId)`
4. [1,25] Uma função tem que obrigatoriamente ter na assinatura a palavra reservada *async*, se:
 - A. no corpo da função for usada a palavra reservada *await*.
 - B. a função chamar outra função que também é *async*.
 - C. se não tiver como último argumento uma função de *callback*
 - D. retornar uma *Promise*
5. [1,25] Uma aplicação *web* recebe um pedido que não pode processar, uma vez que deteta a falta de um parâmetro obrigatório na *query string*. O *status code* adequado para responder a esta situação é:
 - A. 200
 - B. 300
 - C. 400
 - D. 500
6. [1,25] Num formulário HTML (criado com o elemento *form*) com submissão automática do pedido pelo botão de *submit* (definido como elemento integrante do formulário), este botão de submissão é definido com:
 - A. elemento *submit* com atributo *type* de valor *button* (e *method* no *form* com valor *post*)
 - B. elemento *input* com atributo *type* de valor *submit* (e *method* no *form* com valor *post*)
 - C. elemento *request* com atributo *type* de valor *post* (e *method* no *form* com valor *submit*)
 - D. elemento *button* com atributo *type* de valor *post* (e *method* no *form* com valor *submit*)

Instituto Superior de Engenharia de Lisboa
Licenciatura em Engenharia Informática e de Computadores
Licenciatura em Engenharia Informática, Redes e Telecomunicações
Introdução à Programação na Web / Programação na Internet
Inverno de 2021/2022
Teste Global de Época Normal (8 de fevereiro de 2022) - Duração 1h30

NÚMERO _____ NOME _____

GRUPO 1 [7,5 valores]

Nas seguintes questões, selecione a **única** opção verdadeira. Uma resposta correta conta 1,25 valores; uma resposta incorreta desconta 0,25 valores ao total do grupo; uma questão não respondida conta 0 valores.

1. [1,25] Considere o programa em JavaScript apresentado ao lado, que é executado via Node.js.
Aproximadamente quantos segundos decorrem entre a afiação na consola de START e DONE?
A. 2 ou menos segundos
B. 3 segundos
C. 4 segundos
D. 5 ou mais segundos
- ```
let value = 0;
setTimeout(() => { value = 1 }, 1000);
console.log("START");
setTimeout(() => { value = 2 }, 3000);
while (value != 2) /* do nothing */ ;
console.log("DONE!");
```
2. [1,25] Numa aplicação *web*, construída em Node.js utilizando o módulo *express*, existem recursos designados como *item*, identificados por URIs com path `/items/_id_`, em que `_id_` é o identificador de cada item. Pretende-se aceitar um pedido HTTP para criar um novo recurso *item*. Qual das seguintes configurações da aplicação *express* (app) é inaceitável de acordo com a especificação HTTP?  
A. `app.post("/items", createItemWithId)`  
B. `app.post("/items/:id", createItemWithId)`  
C. `app.get("/item/:id/create", createItemWithId)`  
D. `app.put("/items/:id", createItemWithId)`
  3. [1,25] Uma função tem que obrigatoriamente ter na assinatura a palavra reservada *async*, se:  
A. a função chamar outra função que também é *async*.  
B. se não tiver como último argumento uma função de *callback*  
C. retornar uma *Promise*  
D. no corpo da função for usada a palavra reservada *await*.
  4. [1,25] Uma aplicação *web* recebe um pedido que não pode processar, uma vez que deteta a falta de um parâmetro obrigatório na *query string*. O *status code* adequado para responder a esta situação é:  
A. 200  
B. 300  
C. 400  
D. 500
  5. [1,25] Num formulário HTML (criado com o elemento *form*) com submissão automática do pedido pelo botão de *submit* (definido como elemento integrante do formulário), este botão de submissão é definido com:  
A. elemento *input* com atributo *type* de valor *submit* (e *method* no *form* com valor *post*)  
B. elemento *request* com atributo *type* de valor *post* (e *method* no *form* com valor *submit*)  
C. elemento *button* com atributo *type* de valor *post* (e *method* no *form* com valor *submit*)  
D. elemento *submit* com atributo *type* de valor *button* (e *method* no *form* com valor *post*)
  6. [1,25] Em código JavaScript executado pelo *browser* no âmbito de um documento HTML, a forma correcta de obter referências para os elementos cujo atributo *class* inclua *note* é:  
A. `document.querySelectorAll(".note")`  
B. `document.querySelector("class=note")`  
C. `document.querySelector("#note")`  
D. `document.querySelectorNote("class")`

Instituto Superior de Engenharia de Lisboa  
Licenciatura em Engenharia Informática e de Computadores  
Licenciatura em Engenharia Informática, Redes e Telecomunicações  
**Introdução à Programação na Web / Programação na Internet**  
Inverno de 2021/2022  
Teste Global de Época Normal (8 de fevereiro de 2022) - Duração 1h30

---

NÚMERO \_\_\_\_\_ NOME \_\_\_\_\_

**GRUPO 1 [7,5 valores]**

Nas seguintes questões, selecione a **única** opção verdadeira. Uma resposta correta conta 1,25 valores; uma resposta incorreta desconta 0,25 valores ao total do grupo; uma questão não respondida conta 0 valores.

1. [1,25] Uma função tem que obrigatoriamente ter na assinatura a palavra reservada `async`, se:
  - A. se não tiver como último argumento uma função de *callback*
  - B. retornar uma *Promise*
  - C. no corpo da função for usada a palavra reservada `await`.
  - D. a função chamar outra função que também é `async`.
2. [1,25] Uma aplicação *web* recebe um pedido que não pode processar, uma vez que deteta a falta de um parâmetro obrigatório na *query string*. O *status code* adequado para responder a esta situação é:
  - A. 200
  - B. 300
  - C. 400
  - D. 500
3. [1,25] Num formulário HTML (criado com o elemento `form`) com submissão automática do pedido pelo botão de *submit* (definido como elemento integrante do formulário), este botão de submissão é definido com:
  - A. elemento `request` com atributo `type` de valor `post` (e `method` no `form` com valor `submit`)
  - B. elemento `button` com atributo `type` de valor `post` (e `method` no `form` com valor `submit`)
  - C. elemento `submit` com atributo `type` de valor `button` (e `method` no `form` com valor `post`)
  - D. elemento `input` com atributo `type` de valor `submit` (e `method` no `form` com valor `post`)
4. [1,25] Em código JavaScript executado pelo *browser* no âmbito de um documento HTML, a forma correcta de obter referências para os elementos cujo atributo `class` inclua `note` é:
  - A. `document.querySelector("class=note")`
  - B. `document.querySelector("#note")`
  - C. `document.querySelectorNote("class")`
  - D. `document.querySelectorAll(".note")`
5. [1,25] Considere o programa em JavaScript apresentado ao lado, que é executado via *Node.js*. Aproximadamente quantos segundos decorrem entre a afixação na consola de `START` e `DONE`?

```
let value = 0;
setTimeout(() => { value = 1 }, 1000);
console.log("START");
setTimeout(() => { value = 2 }, 3000);
while (value !== 2) /* do nothing */ ;
console.log("DONE!");
```

  - A. 2 ou menos segundos
  - B. 3 segundos
  - C. 4 segundos
  - D. 5 ou mais segundos
6. [1,25] Numa aplicação *web*, construída em *Node.js* utilizando o módulo *express*, existem recursos designados como *item*, identificados por URIs com path `/items/_id_`, em que `_id_` é o identificador de cada item. Pretende-se aceitar um pedido HTTP para criar um novo recurso *item*. Qual das seguintes configurações da aplicação *express* (app) é inaceitável de acordo com a especificação HTTP?
  - A. `app.post("/items/:id", createItemWithId)`
  - B. `app.get("/item/:id/create", createItemWithId)`
  - C. `app.put("/items/:id", createItemWithId)`
  - D. `app.post("/items", createItemWithId)`

**GRUPO 2 [5 valores]**

7. [2] Considere a aplicação *express* definida na listagem seguinte e o erro dado no servidor quando é submetido o formulário. Identifique e justifique a razão do erro.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>const express = require('express') const app = express()  app.get('/hello', (req, res) =&gt; res.send(`   &lt;html&gt;&lt;body&gt;&lt;form method="post"&gt;     Nickname: &lt;input type="text" name="nick"&gt;     &lt;input type="submit"&gt;   &lt;/form&gt;&lt;/body&gt;&lt;/html&gt; `)) app.post('/hello', (req, res) =&gt; {   console.log(req.body.nick)   res.end() }) app.listen(3000, () =&gt; console.log('Listening...'))</pre> | <div><div>Nickname: <input type="text" value="master"/> <input type="submit" value="Submit"/></div><div>Na submissão do formulário o servidor dá o erro:<br/><br/>TypeError: Cannot read properties of undefined (reading 'nick')</div></div> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

8. [3] Considere uma aplicação *express* com as rotas definidas na listagem seguinte e a sequência de pedidos que produz as vistas apresentadas: 1) GET /degrees/leic, 2) GET /degrees/leic/course, 3) POST /degrees/leic/course. Indique em que situação o *browser* produz o aviso apresentado na última imagem e altere a implementação da aplicação *express* de modo a que esta situação não ocorra.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>const degrees = {leic: ['ipw', 'ave', 'pc']}  app.get('/degrees/:deg', (req, res) =&gt; {   res.render('courses', {     courses: degrees[req.params.deg]   }) }) app.get('/degrees/:deg/course', (req, res) =&gt; res.end(`   &lt;html&gt;&lt;body&gt;&lt;form method="post"&gt;     Course: &lt;input type="text" name="course"&gt;     &lt;input type="submit"&gt;   &lt;/form&gt;&lt;/body&gt;&lt;/html&gt; `)) app.post('/degrees/:deg/course', (req, res) =&gt; {   degrees[req.params.deg].push(req.body.course)   res.render('courses', {     courses: degrees[req.params.deg]   }) })</pre> | <div><div><div>localhost:3000/degrees/leic</div><div><b>Courses:</b><ul style="list-style-type: none"><li>ipw</li><li>ave</li><li>pc</li></ul></div></div><div><div>localhost:3000/degrees/leic/course</div><div>Course: <input type="text" value="daw"/> <input type="submit" value="Submit"/></div></div><div><div>localhost:3000/degrees/leic/course</div><div><b>Courses:</b><ul style="list-style-type: none"><li>ipw</li><li>ave</li><li>pc</li><li>daw</li></ul></div><div><b>Resubmit the form?</b><p>The page you're looking for used information that you entered. Returning to that page might trigger a repetition of any action you took there. Do you want to continue?</p><div><input type="button" value="Continue"/> <input type="button" value="Cancel"/></div></div></div></div> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### GRUPO 3 [3 valores]

9. [3] Implemente a função `validateProperty(obj, propValidator)` que recebe em `obj` um objeto e em `propValidator` uma especificação de um validador. O validador é um objeto com duas propriedades: `name` e `validators`, em que `name` é uma *string* com o nome da propriedade de `obj` a validar e `validators` é um *array* de funções de validação (predicados), com o seguinte descritor funcional: `(val) -> Boolean`. A função `validateProperty` verifica se em `obj` existe uma propriedade com o nome `propValidator.name` e, caso exista, se é válida. A validação é realizada chamando as funções presentes em `propValidator.validators`, passando a cada uma o valor da propriedade. A função retorna `true` se a propriedade existir e se todas as funções de validação retornarem `true`; caso contrário retorna `false`.

Exemplos:

```
validateProperty({ a: 123, b: "abc" }, { name: "b", validators: [p => p[0] === "a", p => p.length > 2] }) ⇨ true
validateProperty({ a: 123, b: "abc" }, { name: "b", validators: [p => p[0] === "z", p => p.length > 2] }) ⇨ false
validateProperty({ a: 123, b: "abc" }, { name: "c", validators: [p => p[0] === "a", p => p.length > 2] }) ⇨ false
```

### GRUPO 4 [4,5 valores]

10. [1,5] Considere o seguinte código que usa *promises* e reescreva a função `f1()` para tirar partido da sintaxe `async/await`, sem usar explicitamente os métodos de *Promise*.

```
function f1(p1, p2) {
 return p1.then(v1 => p2.then(v2 => v1 + v2))
}
```

11. [3] Considere a aplicação BORGa desenvolvida no trabalho prático. Pretende-se adicionar a funcionalidade de remover vários grupos de uma única vez. Implemente, no módulo `borga-services`, a função `removeUserGroups(userId, groupsIds)`, que recebe em `userId` o identificador do utilizador, em `groupsIds` os identificadores dos grupos a remover e retorna uma *promise* que fica resolvida quando a operação estiver concluída. A implementação da função `removeUserGroups` chama várias vezes a função `removeGroup(userId, groupId)` do módulo `borga-data`, que recebe em `userId` o identificador do utilizador, em `groupId` o identificador de um dos grupos a remover e retorna uma *promise* que fica resolvida com sucesso quando o grupo estiver removido da base de dados. **Valorizam-se soluções que façam as remoções em paralelo.**

NOTA: Na implementação considere apenas as situações de sucesso, ignorando as validações de `userId` e `groupId`. Assuma também que `removeGroup(userId, groupId)` já está implementada.